《Position Based Dynamics》- 论文阅读笔记

《Position Based Dynamics》

一、Algorithm

1.1 Overview

We represent a dynamic object by a set of N vertices and M constraints. A vertex $i \in [1,\ldots,N]$ has a mass m_i, a position $\mathbf{x}_i$ and a velocity $\mathbf{v}_i$.

A constraint $j \in [1,\ldots,M]$ consists of

- a cardinality n_j,
- a function $C_j : \mathbb{R}^{3n_j} \rightarrow \mathbb{R}$,
- a set of indices $\{i_1,\ldots i_{n_j}\}$, $i_k \in [1,\ldots N]$,
- a stiffness parameter $k_j \in [0\ldots 1]$ and
- a type of either *equality* or *inequality*.

Constraint j with type *equality* is satisfied if $C_j(\mathbf{x}_{i_1},\ldots,\mathbf{x}_{i_{n_j}}) = 0$. If its type is *inequality* then it is satisfied if $C_j(\mathbf{x}_{i_1},\ldots,\mathbf{x}_{i_{n_j}}) \geq 0$. The stiffness parameter k_j defines the strength of the constraint in a range from zero to one.

Based on this data and a time step Δt, the dynamic object is simulated as follows:

(1) **forall** vertices i
(2) initialize $\mathbf{x}_i = \mathbf{x}_i^0, \mathbf{v}_i = \mathbf{v}_i^0, w_i = 1/m_i$
(3) **endfor**
(4) **loop**
(5) **forall** vertices i **do** $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{ext}(\mathbf{x}_i)$
(6) dampVelocities($\mathbf{v}_1,\ldots,\mathbf{v}_N$)
(7) **forall** vertices i **do** $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$
(8) **forall** vertices i **do** generateCollisionConstraints($\mathbf{x}_i \rightarrow \mathbf{p}_i$)
(9) **loop** solverIterations **times**
(10) projectConstraints($C_1,\ldots,C_{M+M_{coll}},\mathbf{p}_1,\ldots,\mathbf{p}_N$)
(11) **endloop**
(12) **forall** vertices i
(13) $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i)/\Delta t$
(14) $\mathbf{x}_i \leftarrow \mathbf{p}_i$
(15) **endfor**
(16) velocityUpdate($\mathbf{v}_1,\ldots,\mathbf{v}_N$)
(17) **endloop**

1.2 算法分析

二、引用的数学理论知识

2.1 Integration

Integration Basics

2.1.1 Explicit

2.1.2 Implicit

2.1.3 Verlet

Verlet Integration

2.2 Iterative method

> [1]. University of Notre Dame： The Jacobi and Gauss-Seidel Iterative Methods

2.2.1 Jacobi

The Jacobi Method

***Two assumptions made on Jacobi Method*:**

1. The system given by

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots a_{nn}x_n &= b_n \end{aligned}$$

Has a unique solution.

2. The coefficient matrix A has no zeros on its main diagonal, namely, $a_{11}, a_{22}, \dots, a_{nn}$ are nonzeros.

<u>Main idea of Jacobi</u>

To begin, solve the 1st equation for x_1, the 2nd equation for x_2 and so on to obtain the rewritten equations:

$$\begin{aligned} x_1 &= \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3 - \cdots a_{1n}x_n) \\ x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3 - \cdots a_{2n}x_n) \\ &\vdots \\ x_n &= \frac{1}{a_{nn}}(b_n - a_{n1}x_1 - a_{n2}x_2 - \cdots a_{n,n-1}x_{n-1}) \end{aligned}$$

Then make an initial guess of the solution $\boldsymbol{x}^{(0)} = (x_1^{(0)}, x_2^{(0)}, x_3^{(0)}, \dots x_n^{(0)})$. Substitute these values into the right hand side the of the rewritten equations to obtain the *first approximation*, $\left(x_1^{(1)}, x_2^{(1)}, x_3^{(1)}, \dots x_n^{(1)}\right)$.

This accomplishes one **iteration**.

In the same way, the *second approximation* $\left(x_1^{(2)}, x_2^{(2)}, x_3^{(2)}, \dots x_n^{(2)}\right)$ is computed by substituting the first approximation's x-vales into the right hand side of the rewritten equations.

By repeated iterations, we form a sequence of approximations $\boldsymbol{x}^{(k)} = \left(x_1^{(k)}, x_2^{(k)}, x_3^{(k)}, \dots x_n^{(k)}\right)^t, \quad k = 1,2,3, \dots$

<u>The Jacobi Method.</u> For each $k \geq 1$, generate the components $x_i^{(k)}$ of $\boldsymbol{x}^{(k)}$ from $\boldsymbol{x}^{(k-1)}$ by

$$x_i^{(k)} = \frac{1}{a_{ii}}\left[\sum_{\substack{j=1, \\ j\neq i}}^{n} (-a_{ij}x_j^{(k-1)}) + b_i\right], \qquad \text{for } i = 1,2,, \dots n$$

Jacobi 迭代法的线性方程组表述形式

The Jacobi Method in Matrix Form

Consider to solve an $n \times n$ size system of linear equations $A\boldsymbol{x} = \boldsymbol{b}$ with $A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$ and $\boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$ for $\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$.

We split A into

$$A = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix} - \begin{bmatrix} 0 & \cdots & 0 & 0 \\ -a_{21} & \cdots & 0 & 0 \\ \vdots & & \ddots & \vdots \\ -a_{n1} & \cdots & -a_{n,n-1} & 0 \end{bmatrix} - \begin{bmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ 0 & 0 & & \vdots \\ \vdots & \vdots & \ddots & -a_{n-1,n} \\ 0 & 0 & \cdots & 0 \end{bmatrix} = D - L - U$$

$A\boldsymbol{x} = \boldsymbol{b}$ is transformed into $(D - L - U)\boldsymbol{x} = \boldsymbol{b}$

$$D\boldsymbol{x} = (L + U)\boldsymbol{x} + \boldsymbol{b}$$

Assume D^{-1} exists and $D^{-1} = \begin{bmatrix} \frac{1}{a_{11}} & 0 & \cdots & 0 \\ 0 & \frac{1}{a_{22}} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{a_{nn}} \end{bmatrix}$

Then

$$\boldsymbol{x} = D^{-1}(L + U)\boldsymbol{x} + D^{-1}\boldsymbol{b}$$

The matrix form of Jacobi iterative method is

$$\boldsymbol{x}^{(k)} = D^{-1}(L + U)\boldsymbol{x}^{(k-1)} + D^{-1}\boldsymbol{b} \qquad k = 1,2,3, \ldots$$

Define $T = D^{-1}(L + U)$ and $\boldsymbol{c} = D^{-1}\boldsymbol{b}$, Jacobi iteration method can also be written as

$$\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{(k-1)} + \boldsymbol{c} \qquad k = 1,2,3, \ldots$$

Jacobi 迭代法的矩阵表述形式

Numerical Algorithm of Jacobi Method

Input: $A = [a_{ij}]$, $\boldsymbol{b}, \boldsymbol{XO} = \boldsymbol{x}^{(0)}$, tolerance TOL, maximum number of iterations N.

Step 1 Set $k = 1$

Step 2 while ($k \leq N$) do Steps 3-6

Step 3 For for $i = 1,2,, \dots n$

$$x_i = \frac{1}{a_{ii}}\left[\sum_{\substack{j=1,\\ j\neq i}}^{n}(-a_{ij}\boldsymbol{XO}_j) + b_i\right],$$

Step 4 If $||\boldsymbol{x} - \boldsymbol{XO}|| < TOL$, then OUTPUT $(x_1, x_2, x_3, \dots x_n)$;

STOP.

Step 5 Set $k = k + 1$.

Step 6 For for $i = 1,2,, \dots n$

Set $\boldsymbol{XO}_i = x_i$.

Step 7 OUTPUT $(x_1, x_2, x_3, \dots x_n)$;

STOP.

Another stopping criterion in Step 4: $\frac{||\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}||}{||\boldsymbol{x}^{(k)}||}$

Jacobi 迭代法的算法伪代码

总结：

1. 从上面不难看到，Jacobi 迭代法针对第 k 次迭代后得到的 X(i)，必须得等到第 k+1 次迭代结束时，才能得到修正，简而言之其收敛速度较慢，并且其收敛性和准确度受到迭代次数的影响
2. Jacobi 迭代需要保证系数矩阵是对角绝对占优（对角线元素绝对值大于该行其他元素绝对值之和，即可逆），才能保证迭代最终是收敛的

2.2.2 Gauss-Seidel

The Gauss-Seidel Method

<u>Main idea of Gauss-Seidel</u>

With the Jacobi method, the values of $x_i^{(k)}$ obtained in the kth iteration remain unchanged until the entire $(k+1)$th iteration has been calculated. With the Gauss-Seidel method, we use the new values $x_i^{(k+1)}$ as soon as they are known. For example, once we have computed $x_1^{(k+1)}$ from the first equation, its value is then used in the second equation to obtain the new $x_2^{(k+1)}$, and so on.

Example. Derive iteration equations for the Jacobi method and Gauss-Seidel method to solve

$$5x_1 - 2x_2 + 3x_n = -1$$
$$-3x_1 + 9x_2 + x_n = 2$$
$$2x_1 - x_2 - 7x_n = 3$$

<u>The Gauss-Seidel Method.</u> For each $k \geq 1$, generate the components $x_i^{(k)}$ of $\boldsymbol{x}^{(k)}$ from $\boldsymbol{x}^{(k-1)}$ by

$$x_i^{(k)} = \frac{1}{a_{ii}}\left[-\sum_{j=1}^{i-1}(a_{ij}x_j^{(k)}) - \sum_{j=i+1}^{n}(a_{ij}x_j^{(k-1)}) + b_i\right], \qquad \text{for } i = 1,2,,\dots n$$

Namely,

$$a_{11}x_1^{(k)} = -a_{12}x_2^{(k-1)} - \cdots - a_{1n}x_n^{(k-1)} + b_1$$
$$a_{21}x_1^{(k)} + a_{22}x_2^{(k)} = -a_{23}x_3^{(k-1)} - \cdots - a_{2n}x_n^{(k-1)} + b_2$$
$$\vdots$$
$$a_{n1}x_1^{(k)} + a_{n2}x_2^{(k)} + \cdots a_{nn}x_n^{(k)} = b_n$$

Gauss-Seidel 迭代法的线性方程组表述形式

$$a_{n1}x_1 + a_{n2}x_2 + \cdots a_{nn}x_n = b_n$$

Matrix form of Gauss-Seidel method.

$$(D-L)\boldsymbol{x}^{(k)} = U\boldsymbol{x}^{(k-1)} + \boldsymbol{b}$$
$$\boldsymbol{x}^{(k)} = (D-L)^{-1}U\boldsymbol{x}^{(k-1)} + (D-L)^{-1}\boldsymbol{b}$$

Define $T_g = (D-L)^{-1}U$ and $\boldsymbol{c}_g = (D-L)^{-1}\boldsymbol{b}$, Gauss-Seidel method can be written as

$$\boldsymbol{x}^{(k)} = T_g\boldsymbol{x}^{(k-1)} + \boldsymbol{c}_g \qquad k = 1,2,3,\dots$$

Gauss-Seidel 迭代法的矩阵表述形式

Numerical Algorithm of Gauss-Seidel Method

Input: $A = [a_{ij}]$, $\boldsymbol{b}, \boldsymbol{XO} = \boldsymbol{x}^{(0)}$, tolerance TOL, maximum number of iterations N.

Step 1 Set $k = 1$

Step 2 while ($k \leq N$) do Steps 3-6

Step 3 For for $i = 1,2,, \dots n$

$$x_i = \frac{1}{a_{ii}}\left[-\sum_{j=1}^{i-1}(a_{ij}x_j) - \sum_{j=i+1}^{n}(a_{ij}\boldsymbol{XO}_j) + b_i\right],$$

Step 4 If $||\boldsymbol{x} - \boldsymbol{XO}|| < TOL$, then OUTPUT $(x_1, x_2, x_3, \dots x_n)$;

STOP.

Step 5 Set $k = k + 1$.

Step 6 For for $i = 1,2,, \dots n$

Set $\boldsymbol{XO}_i = x_i$.

Step 7 OUTPUT $(x_1, x_2, x_3, \dots x_n)$;

STOP.

Gauss-Seidel 迭代法的算法伪代码

总结：

1. Gauss-Seidel 迭代法会在每次迭代中都能及时更新修正当前状态，理论上，只要线性方程组有解，其收敛速度会远高于 Jacobi
2. Gauss-Seidel 迭代不需要像 Jacobi 那样系数矩阵必须对角占优，因此求解要求不高，换言之相比 Jacobi 最终迭代能收敛的会更多

2.2.3 Jacobi vs Gauss-Seidel

三阶方程验证：

Jacobi 在第 14 次收敛

Gauss-Seidel 在第 5 次就收敛了

```
Jacobi Iteration:
[1] -0.143, 0.25, -0.231
[2] 0.028, 0.225, -0.415
[3] 0.099, 0.312, -0.216
[4] 0.039, 0.314, -0.141
[5] 0.007, 0.282, -0.21
[6] 0.028, 0.279, -0.244
[7] 0.041, 0.291, -0.22
[8] 0.035, 0.293, -0.206
[9] 0.029, 0.289, -0.213
[10] 0.031, 0.288, -0.22
[11] 0.034, 0.289, -0.217
[12] 0.033, 0.29, -0.214
[13] 0.032, 0.289, -0.215
[14] 0.032, 0.289, -0.216
[15] 0.032, 0.289, -0.216

Gauss-Seidel Iteration:
[1] -0.143, 0.196, -0.411
[2] 0.089, 0.335, -0.154
[3] 0.019, 0.276, -0.23
[4] 0.035, 0.292, -0.213
[5] 0.032, 0.289, -0.216
[6] 0.032, 0.289, -0.216
[7] 0.032, 0.289, -0.216
[8] 0.032, 0.289, -0.216
[9] 0.032, 0.289, -0.216
[10] 0.032, 0.289, -0.216
[11] 0.032, 0.289, -0.216
[12] 0.032, 0.289, -0.216
[13] 0.032, 0.289, -0.216
[14] 0.032, 0.289, -0.216
[15] 0.032, 0.289, -0.216
```

四阶方程验证：

Jacobi 无法收敛（甚至迭代 10w 次也无法收敛）

测试用例中线性方程组的系数矩阵不是对角占优

Gauss-Seidel 在第 9 次就收敛了

```
Standard Result:
-0.723, -0.166, -1.217, 0.847

Jacobi Iteration:
[1] -0.2, 0.222, -0.429, 0.444
[2] -0.475, 0.055, -0.835, 0.498
[3] -0.374, -0.009, -0.928, 0.66
[4] -0.571, -0.019, -1.006, 0.818
[5] -0.749, -0.129, -1.173, 0.743
[6] -0.588, -0.145, -1.155, 0.784
[7] -0.663, -0.107, -1.136, 0.887
[8] -0.803, -0.168, -1.236, 0.807
[9] -0.655, -0.177, -1.21, 0.808
[10] -0.676, -0.131, -1.168, 0.893
[11] -0.802, -0.171, -1.241, 0.831
[12] -0.687, -0.184, -1.227, 0.813
[13] -0.676, -0.141, -1.179, 0.886
[14] -0.789, -0.167, -1.234, 0.842
[15] -0.705, -0.184, -1.232, 0.815
[16] -0.675, -0.148, -1.186, 0.878
[17] -0.777, -0.164, -1.227, 0.851
[18] -0.721, -0.184, -1.235, 0.817
[19] -0.676, -0.153, -1.192, 0.869
[20] -0.763, -0.16, -1.221, 0.856
[21] -0.73, -0.182, -1.235, 0.821
[22] -0.681, -0.158, -1.198, 0.862
[23] -0.751, -0.159, -1.216, 0.859
[24] -0.737, -0.179, -1.234, 0.825
[25] -0.686, -0.161, -1.203, 0.855
[26] -0.74, -0.158, -1.212, 0.861
[27] -0.741, -0.177, -1.232, 0.829
[28] -0.692, -0.164, -1.207, 0.85
[29] -0.731, -0.158, -1.21, 0.861
[30] -0.743, -0.174, -1.23, 0.834
[31] -0.699, -0.167, -1.212, 0.846
[32] -0.724, -0.158, -1.209, 0.861
[33] -0.743, -0.172, -1.228, 0.838
[34] -0.705, -0.168, -1.215, 0.844
[35] -0.72, -0.159, -1.209, 0.859
[36] -0.741, -0.17, -1.225, 0.841
[37] -0.71, -0.169, -1.217, 0.843
[38] -0.718, -0.16, -1.209, 0.858
[39] -0.74, -0.169, -1.224, 0.843
[40] -0.713, -0.169, -1.218, 0.842
[41] -0.716, -0.161, -1.21, 0.856
[42] -0.737, -0.167, -1.222, 0.845
[43] -0.717, -0.169, -1.219, 0.842
[44] -0.715, -0.162, -1.211, 0.854
[45] -0.734, -0.166, -1.22, 0.847
[46] -0.72, -0.169, -1.22, 0.842
[47] -0.714, -0.163, -1.212, 0.853
[48] -0.732, -0.165, -1.219, 0.849
[49] -0.723, -0.169, -1.221, 0.842
[50] -0.714, -0.164, -1.212, 0.852

Gauss-Seidel Iteration:
[1] -0.2, 0.156, -0.508, 0.58
[2] -0.645, -0.13, -1.089, 0.745
[3] -0.69, -0.144, -1.137, 0.806
[4] -0.704, -0.155, -1.183, 0.833
[5] -0.718, -0.163, -1.205, 0.842
[6] -0.721, -0.165, -1.212, 0.845
[7] -0.722, -0.165, -1.215, 0.846
[8] -0.721, -0.165, -1.215, 0.847
[9] -0.723, -0.166, -1.216, 0.847
[10] -0.723, -0.166, -1.216, 0.847
```

```
[99970] -0.727, -0.167, -1.219, 0.845
[99971] -0.718, -0.166, -1.216, 0.847
[99972] -0.723, -0.164, -1.215, 0.85
[99973] -0.727, -0.167, -1.219, 0.845
[99974] -0.718, -0.166, -1.216, 0.847
[99975] -0.723, -0.164, -1.215, 0.85
[99976] -0.727, -0.167, -1.219, 0.845
[99977] -0.718, -0.166, -1.216, 0.847
[99978] -0.723, -0.164, -1.215, 0.85
[99979] -0.727, -0.167, -1.219, 0.845
[99980] -0.718, -0.166, -1.216, 0.847
[99981] -0.723, -0.164, -1.215, 0.85
[99982] -0.727, -0.167, -1.219, 0.845
[99983] -0.718, -0.166, -1.216, 0.847
[99984] -0.723, -0.164, -1.215, 0.85
[99985] -0.727, -0.167, -1.219, 0.845
[99986] -0.718, -0.166, -1.216, 0.847
[99987] -0.723, -0.164, -1.215, 0.85
[99988] -0.727, -0.167, -1.219, 0.845
[99989] -0.718, -0.166, -1.216, 0.847
[99990] -0.723, -0.164, -1.215, 0.85
[99991] -0.727, -0.167, -1.219, 0.845
[99992] -0.718, -0.166, -1.216, 0.847
[99993] -0.723, -0.164, -1.215, 0.85
[99994] -0.727, -0.167, -1.219, 0.845
[99995] -0.718, -0.166, -1.216, 0.847
[99996] -0.723, -0.164, -1.215, 0.85
[99997] -0.727, -0.167, -1.219, 0.845
[99998] -0.718, -0.166, -1.216, 0.847
[99999] -0.723, -0.164, -1.215, 0.85
[100000] -0.727, -0.167, -1.219, 0.845

Gauss-Seidel Iteration:
[1] -0.2, 0.156, -0.508, 0.58
[2] -0.645, -0.13, -1.009, 0.745
[3] -0.69, -0.144, -1.137, 0.806
[4] -0.704, -0.155, -1.183, 0.833
[5] -0.718, -0.163, -1.205, 0.842
[6] -0.721, -0.165, -1.212, 0.845
[7] -0.722, -0.165, -1.215, 0.846
[8] -0.721, -0.165, -1.215, 0.847
[9] -0.723, -0.166, -1.216, 0.847
[10] -0.723, -0.166, -1.216, 0.847
```

2.2.4 Newton-Raphson

Newton-Raphson Method